Repaso para el examen

React + TypeScript

Resumen de todo el temario y ejercicios de repaso resueltos

Unidad	Contenido
UT 02.1	Introducción a React: Vite, estructura, JSX, componentes, export/import
UT 02.2	Props, estado (`useState`), eventos y React DevTools
UT 01.1	TypeScript I: tipos básicos, objetos, arrays, funciones, uniones, enum, interfaces, `tsc`
UT 02.3	React con TypeScript: tipado de props, estado y eventos, `any`/`unknown`, `as const`, `tsconfig`
UT 02.4	Estados y eventos en profundidad, formularios controlados, elevar el estado
Parte 2	Ejercicios de repaso resueltos (Interruptor, Contador, Lista de la compra, Buscador)

Cómo usar este PDF

Lee primero la **chuleta de la sección 7** (errores típicos y preguntas de repaso) y vuelve al temario cuando algo no te cuadre. Los ejercicios de la Parte 2 son los que más probablemente caigan en forma de práctica: intenta resolverlos tú antes de mirar la solución.

Índice

1. Introducción a React (UT 02.1) 3
2. Props, estado y eventos (UT 02.2) 6
3. TypeScript I (UT 01.1) 8
4. React con TypeScript (UT 02.3) 10
5. Estados y eventos en profundidad (UT 02.4) 13
6. Errores frecuentes y buenas prácticas 18
7. Preguntas de repaso con respuesta 18
Ejercicio 1. Interruptor de luz 21
Ejercicio 2. Contador con límites 22
Ejercicio 3. Lista de la compra 23
Ejercicio 4. Buscador de películas 25

1. Introducción a React (UT 02.1)

1.1 Qué es React y por qué es popular

- Biblioteca para crear **interfaces de usuario basadas en componentes**. La creó **Meta** (Facebook) y es de código abierto. La usan Netflix, Uber, Airbnb…
- **Sencillez**: se centra en una sola cosa, crear componentes de UI.
- **Componentes reutilizables**: piezas de interfaz que se combinan y se reutilizan en distintas pantallas o aplicaciones.
- **Adopción incremental**: puede incorporarse a una app existente aunque use otro framework (no necesita hacerse cargo de toda la aplicación).
- **Ecosistema y aprendizaje**: comunidad enorme y herramientas como **Vite**.
- **Componentes de servidor**: recuperación de datos de alto rendimiento y menos JavaScript enviado al navegador. **React Native**: apps iOS/Android reutilizando habilidades y código.
- Meta lo usa en Facebook, así que cuida su calidad y que las nuevas versiones sean baratas de adoptar.

1.2 El DOM virtual

Los cambios en el DOM real son costosos. React mantiene una **representación en memoria (DOM virtual)** y sigue 3 pasos:

Paso	Qué ocurre
1. Cambia el estado	Un componente actualiza sus datos (por ejemplo, tras un clic).
2. Genera y compara	React crea un nuevo DOM virtual y lo compara con el actual (**diffing**).
3. Actualiza lo mínimo	Solo se aplican al DOM real los cambios estrictamente necesarios.

1.3 Herramientas y creación del proyecto con Vite

- Herramientas: navegador moderno (Chrome), terminal, **Visual Studio Code**, **Node.js y npm**.

```
npm create vite@latest        # pide nombre, framework (React) y variante (JavaScript / TypeScript)
cd mi-app-react
npm install                   # instala las dependencias (crea node_modules)
npm run dev                   # servidor de desarrollo en http://localhost:5173  (Ctrl + C para parar)

# Atajo para TypeScript sin preguntas:
npm create vite@latest alert -- --template react-ts
```

Archivo / carpeta	Descripción
`node_modules`	Paquetes npm de los que depende el proyecto (se crea con `npm install`).
`public`	Recursos estáticos (imágenes…) servidos en la ruta raíz `/`.
`src/main.jsx`	**Punto de entrada**: carga la app React en `index.html`.
`src/App.jsx`	Componente de nivel superior, llamado `App`.
`src/index.css`, `App.css`	Estilos globales y estilos del componente App.
`index.html`	Página web raíz donde se carga la aplicación React.
`package.json`	Nombre, versión, dependencias y **scripts** (`dev`, `build`…).
`package-lock.json`	Versiones exactas de las dependencias (coherencia entre entornos).
`eslint.config.js`	Configuración de ESLint.
.gitignore / README.md	Qué ignora Git / información de la plantilla.

1.4 Calidad de código: ESLint y Prettier

- **Linting** = revisar el código en busca de posibles problemas mientras se escribe. **ESLint** ya viene instalado y configurado por Vite; se instala además la **extensión ESLint** en VS Code y se activa

`eslint.probe` para los lenguajes (javascript, javascriptreact, y typescript / typescriptreact).

- Se **desactiva la regla de prop-types** porque con TypeScript se tipan las props: en `eslint.config.js`, dentro de `rules`: `'react/prop-types': 'off'`.
- **Prettier** formatea el código automáticamente. Vite NO lo configura: `npm i -D prettier` y `npm i -D eslint-config-prettier` (`-D` = `--save-dev`, dependencia solo de desarrollo). Se añade `prettier` al final del array de `eslint.config.js`.
- Configuración en `.prettierrc.json`:

```
{
  "printWidth": 100,        // líneas de hasta 100 caracteres
  "singleQuote": true,      // comillas simples
  "semi": true,             // punto y coma al final
  "tabWidth": 2,            // sangría de 2 espacios
  "trailingComma": "all",   // coma final en arrays/objetos multilínea
  "endOfLine": "auto"       // mantiene los finales de línea existentes
}
```

- En VS Code: instalar la extensión **Prettier - Code formatter**, activar **Format on Save** y poner Prettier como **Default Formatter** (pestaña Espacio de trabajo).

1.5 Modo desarrollo y compilación de producción

	Desarrollo: `npm run dev`	**Producción:** `npm run build`
Qué hace	Servidor de desarrollo de Vite en `localhost:5173`. Transpila JSX a JavaScript al vuelo y recarga al guardar.	Genera los artefactos en la carpeta `dist` (con `index.html` como archivo principal).
Procesos	—	**Minificación** (quita espacios/comentarios, acorta nombres), **empaquetado** (fusiona archivos en bundles) y **tree-shaking** (elimina código no usado).

1.6 Estructura de la app: main.jsx, JSX y árbol de componentes

```
// src/main.jsx
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

- `createRoot` crea una raíz en el elemento con id `root` del HTML. `render` muestra los componentes indicados (**renderizado**).
- `StrictMode`: componente especial que activa comprobaciones extra y avisos **solo en desarrollo**.
- **JSX** (JavaScript XML): permite escribir código parecido a HTML dentro de JavaScript. Hay que **transpilarlo** a llamadas a funciones JS (con Babel u otra herramienta) antes de ejecutarlo en el navegador.
- Una app React es un **árbol de componentes**. Ejemplo: `StrictMode` → `App` → `Header`, `Main`, `Footer`.
- Un **componente es una función JavaScript normal que devuelve JSX**. El JSX puede contener elementos HTML y otros componentes. Dentro de las llaves `{ }` se mete JavaScript (variables, funciones, expresiones).
- El **fragmento** `<> ... </>` agrupa elementos sin crear un nodo extra en el DOM.

1.7 Crear y reutilizar un componente (Alert)

```
// src/Alert.jsx
export function Alert() {
  return (
    <div>
      <div>
        <span role="img" aria-label="Advertencia">⚠</span>
        <span>¡Oh, no!</span>
      </div>
```

```
      <div>Ha ocurrido un error</div>
    </div>
  );
}

// src/App.jsx
import { Alert } from './Alert';
function App() {
  return <Alert />;
}
```

- El nombre del componente **debe empezar por mayúscula**; si no, React lo trata como un elemento HTML y no se renderiza bien.
- El archivo debe tener extensión `.js` o `.jsx` (en TypeScript: `.tsx`). El nombre del archivo no importa a React; lo habitual es usar el mismo que el componente.
- Cada componente suele ir en **su propio archivo**: hay que **exportarlo** (`export function Alert`) para poder **importarlo** (`import { Alert } from './Alert'`). Exportarlo también arregla el aviso de ESLint de 'componente no utilizado'.
- Función normal o función flecha (`const Alert = () => {...}`): **no hay diferencias significativas** para un componente de función; es cuestión de preferencia.
- `role="img"` y `aria-label` en el emoji ayudan a los **lectores de pantalla** (accesibilidad).

Ojo examen
Nombre en mayúscula, un componente por archivo, `export`/`import`, fragmento `<></>`, JSX se transpila, `StrictMode` solo actúa en desarrollo. Son preguntas típicas.

2. Props, estado y eventos (UT 02.2)

2.1 Props: datos del padre al hijo

- **Props** = *properties*. Es un parámetro **objeto** opcional que recibe el componente; contiene las propiedades que elijamos. Permite que el padre le pase datos al hijo.
- Se pasan **como atributos JSX** y los nombres deben coincidir con los que espera el componente. El parámetro puede llamarse como quieras, pero por convención se llama `props`.
- Las props son de **solo lectura**; hacen la salida del componente flexible.

```
function DatosContacto(props) {
  console.log(props.nombre);
  console.log(props.email);
}
<DatosContacto nombre="Fred" email="fred@ejemplo.com" />
```

Props de Alert

Prop	Para qué sirve
`type`	`'information'` o `'warning'`: decide el icono (y su `aria-label`).
`heading`	Título de la alerta.
`children`	Prop **especial**: el contenido escrito **entre las etiquetas** del componente.
`closable`	Si es verdadera, se muestra el botón de cierre.
`onClose`	Evento propio (función) que se dispara al cerrar.

```
// Con desestructuración y valor por defecto (la forma "limpia")
export function Alert({ type = 'information', heading, children, closable, onClose }) {
  ...
}

// Uso
<Alert type="information" heading="Success" closable>
  ¡Todo va muy bien!
</Alert>
```

- **Desestructurar** el parámetro evita escribir `props.` delante de cada propiedad. El **valor por defecto** (`type = 'information'`) se usa si no se pasa la prop.
- Un **atributo booleano sin valor** equivale a `true`: `closable` es lo mismo que `closable={true}`.

2.2 Estado: datos que cambian (useState)

- El **estado** es una variable especial que puede cambiar durante la vida del componente. **Cuando cambia, React vuelve a renderizar el componente (re-render).** Es la clave para que un componente sea **interactivo**. Hay componentes que no necesitan estado.
- Se define con el **hook** `useState`: `const [state, setState] = useState(initialState);`
- `useState` devuelve una **tupla** (valor actual, función para actualizarlo), que se desestructura. Convención de nombres: `visible` / `setVisible`.
- Si no se pasa valor inicial, empieza como `undefined`. Se pueden tener **varios estados** (varias llamadas a `useState`).

```
import { useState } from 'react';

export function Alert({ type = 'information', heading, children, closable, onClose }) {
  const [visible, setVisible] = useState(true);   // la alerta empieza visible

  if (!visible) {
    return null;                                  // no renderiza nada
  }
  ...
}

const [loading, setLoading] = useState(true);
const [error, setError] = useState();             // undefined al principio
```

Renderizado condicional con &&

```
{closable && (
  <button aria-label="Cerrar" onClick={handleCloseClick}>
    <span role="img" aria-label="Cerrar">□</span>
  </button>
)}
```

- Si `closable` es **falso** (falsy), la expresión se **interrumpe** y no se pinta el botón; si es **verdadero** (truthy), se muestra.

2.3 Eventos

- Los eventos del navegador se producen cuando el usuario interactúa con el DOM (clic, teclado...). Los eventos de React son una **capa que envuelve los eventos nativos**.
- Un **controlador de eventos** (*event handler*) es la función que se ejecuta cuando ocurre el evento. Se registra en JSX con un atributo en **camelCase**: `onClick={handleCloseClick}`.
- Se pasa **la función, no su resultado**: `onClick={handleClick}` ✓ — `onClick={handleClick()}` ✗ (se ejecutaría al renderizar).
- El manejador se define **dentro** del componente para tener acceso a props y estado. Puede ser función declarada, función flecha o **en línea**: `onClick={() => {...}}`.

```
function handleCloseClick() {
  setVisible(false);          // 1) cambia el estado -> re-render y desaparece
  if (onClose) {              // 2) evento propio: solo si el consumidor lo pasó
    onClose();
  }
}

// En App.jsx (consumidor): manejador en línea
<Alert type="information" heading="Success" closable
       onClose={() => console.log('closed')}>
  ¡Todo va muy bien!
</Alert>
```

- **Evento personalizado** = una **prop de función** que el componente invoca. Por convención su nombre empieza por `on` (`onClose`). Solo se llama si está definida, así no se obliga al consumidor a gestionarlo.

2.4 Componentes de función y herramientas de desarrollo

- Se prefieren los **componentes de función** frente a los de clase porque: requieren **menos código**, la **lógica es más fácil de reutilizar** y los **Hooks no se pueden usar en componentes de clase**.

React DevTools	Para qué sirve
Extensión de navegador	Chrome, Firefox y Edge. Se abre con **F12** (Windows) o **Cmd + Opción + I** (Mac): paneles **Components** y **Profiler**.
Panel **Components**	Muestra el árbol de componentes, sus **props** y **estado**. Puedes **editar** props y estado y ver el resultado al instante. El icono de la varita (*Parse hook names*) muestra el nombre de las variables de estado (`State(visible)`).
Panel **Profiler**	Perfila interacciones para detectar componentes lentos. Pasos: activar *Record why each component rendered*, **Iniciar** perfilado (círculo azul), interactuar (clic en cerrar), **Detener** (círculo rojo). Muestra la línea de tiempo (la alerta se re-renderizó en 0,7 ms).

3. TypeScript I (UT 01.1)

3.1 Qué es y cómo se usa

- TypeScript es un **superconjunto sintáctico estricto de JavaScript** que añade **tipado estático** (JavaScript es de tipado dinámico). Todo JavaScript es válido en TypeScript, pero **no al revés**.
- Ventaja: el compilador detecta los **errores de tipos al instante**, mientras escribes; en JavaScript solo se detectan en ejecución (por ejemplo `array.map` sobre un número).
- Limitación: **los navegadores solo entienden JavaScript**, así que TypeScript se **compila** (transpila) a JS.
- Instalación global: `npm i -g typescript`. Compilar un archivo: `tsc main.ts` → genera `main.js` en el mismo directorio. `npm -v` comprueba la versión de npm (gestor de paquetes por defecto de Node.js; en npmjs.com cualquiera puede publicar, no hay control de calidad).
- La extensión **Live Server** de VS Code recarga el navegador automáticamente en cada cambio.

3.2 Tipos básicos e inferencia

```
let myNum = 10;               // inferido: number (no hace falta anotar)
let anotherNum: number = 20;  // anotado explícitamente
myNum = '12';                 // Error: string no asignable a number

let myStr: string = 'Hello';  let anotherStr = 'World';   // string
let myBool: boolean = true;   let anotherBool = false;    // boolean

// Inferencia: ¿cuándo anotar? Si vas a asignar el valor MÁS ADELANTE, especifica el tipo
let salary;                   // sin tipo -> acepta cualquier cosa (any)  ✗
let salary2: number;          // con tipo -> salary2 = '12000' da error   ✓
```

- Regla: **deja que TypeScript infiera** el tipo; anota explícitamente cuando declaras sin valor inicial o cuando la inferencia no da el tipo deseado.

3.3 Objetos, arrays y funciones

```
// Objeto con tipo por clave: error si asignas otro tipo o una clave que no existe
const newDeveloper: { name: string; age: number; isDev: boolean } =
  { name: 'Mousam', age: 39, isDev: true };
newDeveloper.age = 'Forty';          // Error
newDeveloper.firstName = 'Mousam';   // Error: no existe

// Arrays
const languages = ['React', 'Angular', 'Vue'];   // string[]
languages.push(56);                              // Error
const numbers: number[] = [51, 22, 33];
const arrOfObj: { name: string; age: number }[] = [{ name: 'Nabendu', age: 40 }];
const arrOfArrays: number[][] = [[11, 32, 43], [34, 75, 64]];   // doble corchete

// Funciones: tipa SIEMPRE los parámetros; también el tipo de retorno
const multiNums = (num1: number, num2: number) => num1 * num2;
multiNums(10, '20');                                  // Error
const modNums = (a: number, b: number): number => a > b;   // Error: devuelve boolean
const printSum = (a: number, b: number): void => { console.log(a + b); };  // void = no devuelve nada
```

3.4 Unión, literales, enum, opcionales

```
// Unión: varios tipos posibles
let numOrStr: number | string;
let arr: (number | string)[] = [10, 'Ten', true];       // Error: boolean no permitido

// Literal: solo valores exactos
let myLiteral: 'Nabendu' | 'Mousam' | 'Shikha' | 'Hriday' = 'Nabendu';
myLiteral = 'Parag';                                     // Error

// Enum: conjunto fijo de valores con nombre (unión + literal)
enum Role { ADMIN, READ_ONLY, AUTHOR };
const hridayRole: Role = Role.AUTHOR;

// Opcional: ? -> puede omitirse; si se da, debe cumplir el tipo
let betterOptObj: { name: string; age?: number } = { name: 'Nabendu' };
```

```
let optionalObj: { name: string; age: number | undefined } = { name: 'Nabendu', age: undefined };
// (con "| undefined" hay que escribir la clave aunque sea undefined)
```

3.5 Interfaces y type

```
interface Developer { name: string; age: number; isDev: boolean; }
const person1: Developer = { name: 'Nabendu', age: 40, isDev: true };

type DeveloperType = { name: string; age: number; isDev: boolean; };
type PersonName = string;                        // un type puede ser un string...
type CoderType = { name: string; category: 'frontend' | 'backend' | 'mobile'; age: number; }[];
```

Se puede usar en...	interface	type
Objetos	Sí	Sí
Strings / uniones / arrays / funciones	No	Sí
Ampliar otro tipo	`extends`	`&` (intersección)
Declaraciones repetidas que se fusionan	Sí	No (da error)

3.6 El compilador tsc

Qué quiero	Comando / configuración
Modo vigilancia (recompila al guardar)	`tsc main.ts -w`
Convertir la carpeta en proyecto TS	`tsc --init` → crea `tsconfig.json` en la raíz
Compilar todo el proyecto	`tsc` (compila **todos** los `.ts` según `tsconfig.json`)
Un archivo con errores bloquea todo	`"exclude": ["index.ts", "node_modules"]`
Compilar solo ciertos archivos	`"include": ["main.ts"]`
Organizar el proyecto	`.ts` en carpeta `src`, JS generado en `dist`: `"rootDir": "./src"`, `"outDir": "./dist"` (descomentarlos)
Después	Cambiar en `index.html`: `<script src="dist/main.js"></script>`

> **Ojo examen**
> Si pasas un archivo concreto a `tsc` (`tsc src/welcome.ts`) el compilador **ignora** `tsconfig.json`. `exclude` quita, `include` añade, `rootDir` = origen del .ts, `outDir` = destino del .js.

4. React con TypeScript (UT 02.3)

4.1 Ventajas de TypeScript

- **Detección temprana de errores**: p. ej. escribir `product.price` cuando la propiedad real es `unitPrice` se marca con subrayado rojo al instante (en JS pasaría a producción).
- **IntelliSense** mejor: autocompleta las propiedades reales de un objeto, ayuda en refactorizaciones (renombrar componentes) y navegación. Muy útil en bases de código grandes.
- Novedades 2026 citadas en los apuntes: **TypeScript 6.0** (marzo): `strict` activado por defecto, destino por defecto el ECMAScript más reciente, declara obsoletas opciones antiguas. **TypeScript 7.0** (julio): compilador reescrito en **Go**, compilaciones **8–12× más rápidas**, las opciones obsoletas ya dan error.

4.2 Tipos de JavaScript

- JavaScript tiene **tipado dinámico**: el tipo va con el **valor**, no con la variable (`let score = 9; score = 'ten';` es válido). También **tipado débil**: conversiones implícitas (`'5' * 2` → `10`, `'5' + 2` → `'52'`).
- `typeof` solo distingue unos pocos primitivos y devuelve `'object'` para cualquier objeto (incluida una fecha y `null`). Para comprobar fechas: `date instanceof Date`.

4.3 Anotaciones, inferencia, Date y arrays

```
let unitPrice: number;                          // anotación: dos puntos + tipo
function getTotal(unitPrice: number, quantity: number, discount: number): number { ... }
let flag = false;                               // inferido boolean; flag = 'table' -> error
let today = new Date();                         // inferido Date; today.addMonths(2) -> error
const numbers: number[] = [];  /  Array<number>  // dos sintaxis equivalentes
```

- Las anotaciones de tipo **desaparecen** al transpilar (no existen en JavaScript).
- Confía en la **inferencia** y anota solo cuando no da el tipo que quieres.

4.4 any, unknown y never

Tipo	Qué es
`any`	**Desactiva la comprobación de tipos.** Se asigna a una variable declarada sin tipo ni valor (`let flag;`). **Evítalo.**
`unknown`	Acepta cualquier valor pero **no deja hacer nada con él** hasta comprobar qué es (**estrechar el tipo**, *type narrowing*). Alternativa **segura** a `any`.
`never`	Tipo que **no admite ningún valor** (lo contrario de `unknown`). Ej.: función que siempre lanza un error: `function lanzarError(m: string): never { throw new Error(m); }`

```
fetch('https://swapi.dev/api/people/1')
  .then((response) => response.json())
  .then((data: unknown) => {
    if (isCharacter(data)) {          // dentro del if, data pasa de unknown a { name: string }
      console.log('name', data.name);
    }
  });

// Guarda de tipo (type guard) con predicado de tipo: "character is { name: string }"
function isCharacter(character: unknown): character is { name: string } {
  return (
    typeof character === 'object' &&   // 1) es un objeto
    character !== null &&              // 2) no es null (typeof null === 'object')
    'name' in character &&             // 3) tiene la propiedad name
    typeof character.name === 'string' // 4) y es string
  );
}
```

4.5 Crear tipos propios

```
type Product = { name: string; unitPrice?: number };       // alias de tipo; ? = opcional
type DiscountedProduct = Product & { discount: number };   // & = intersección
```

```
type Purchase = (quantity: number) => void;               // alias para una función
type RGB = 'red' | 'green' | 'blue';                      // unión de literales
type RGBoNulo = 'red' | 'green' | 'blue' | null;          // la unión puede mezclar tipos

interface Alumno { nombre: string; curso: number; }
interface AlumnoErasmus extends Alumno { paisOrigen: string; }   // interface se amplía con extends
type ProfesorTutor = Profesor & { grupoTutorizado: string };     // type se amplía con &
```

- `?` también marca **parámetros opcionales**: `myFunction(requiredParam: string, optionalParam?: string)`.
- En React es habitual usar `type` **para las props** (combina bien con uniones e intersecciones). Lo importante es ser **coherente** en el proyecto.

4.6 as const y satisfies (alternativa moderna a enum)

```
const configuracion = { tema: 'oscuro', idioma: 'es' } as const;
// { readonly tema: 'oscuro'; readonly idioma: 'es' }   -> literales y solo lectura

type Tema = 'claro' | 'oscuro';
const ajustes = { tema: 'oscuro', tamanoFuente: 16 } satisfies { tema: Tema; tamanoFuente: number };
ajustes.tema;   // tipo 'oscuro' -> comprueba el tipo SIN perder el tipo inferido

const Rol = { Admin: 'ADMIN', SoloLectura: 'SOLO_LECTURA', Autor: 'AUTOR' } as const;
type Rol = (typeof Rol)[keyof typeof Rol];     // 'ADMIN' | 'SOLO_LECTURA' | 'AUTOR'
const miRol: Rol = Rol.Admin;
```

- **¿Por qué evitar** `enum`**?** Genera código JavaScript propio (no basta con borrar los tipos al compilar). La opción `erasableSyntaxOnly` (activada en la plantilla de Vite) **prohíbe los enum**.

4.7 El compilador y tsconfig.json

```
{
  "compilerOptions": {
    "outDir": "dist", "target": "esnext", "module": "esnext", "lib": ["DOM", "esnext"],
    "strict": true, "jsx": "react-jsx", "moduleResolution": "bundler", "noEmitOnError": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

Opción	Significado
`outDir`	Carpeta del JavaScript transpilado.
`target` / `module`	Versión de JS de destino / sistema de módulos. `esnext` = lo más reciente.
`lib`	Tipos de la biblioteca estándar: `DOM` (API del navegador) y `esnext`.
`strict`	Comprobación de tipos más estricta (incluye `strictNullChecks` y `noImplicitAny`). Por defecto desde TS 6.
`jsx: react-jsx`	Modo actual de JSX: **no hace falta importar React** en cada archivo.
`moduleResolution: bundler`	Recomendado con Vite. `node` está **obsoleto** (error en TS 7).
`noEmitOnError`	Si hay errores de tipo, **no genera** el JavaScript.

- `npm run build` ejecuta `tsc` con el `tsconfig.json`. Los errores de tipo salen en el terminal.
- **Node.js (22.18+) puede ejecutar** `.ts` **directamente** (`node src/welcome.ts`), pero solo **borra los tipos y NO los comprueba**: para comprobar usa `tsc --noEmit`.
- `"typescript": "*"` instala la última versión (TS 7); para la 6: `npm i -D typescript@6`.

4.8 React + TypeScript: el componente Alert tipado

- Proyecto: `npm create vite@latest alert -- --template react-ts`. Los componentes tienen extensión `.tsx`.
- La plantilla genera **tres** archivos de config: `tsconfig.json` (solo referencias), `tsconfig.app.json` (código de `src`) y `tsconfig.node.json` (`vite.config.ts`).

- `verbatimModuleSyntax` **obliga a importar los tipos con la palabra clave** `type`; si no: error *'ReactNode' is a type and must be imported using a type-only import*.

```
import { useState, type ReactNode } from 'react';       // <- "type" obligatorio para ReactNode

type Props = {
  type?: 'information' | 'warning';   // unión literal: detecta erratas ('warnig') y autocompleta
  heading: string;                    // obligatoria
  children: ReactNode;                // JSX, cadenas, números...  (obligatoria)
  closable?: boolean;                 // opcional
  onClose?: () => void;               // evento propio opcional
};

export function Alert({ type = 'information', heading, children, closable, onClose }: Props) {
  const [visible, setVisible] = useState(true);            // inferido: boolean
  if (!visible) return null;
  function handleCloseClick() {
    setVisible(false);
    onClose?.();                                           // encadenamiento opcional
  }
  ...
}

<Alert />                              // Error: faltan heading y children
<Alert type="error" heading="Fallo">   // Error: 'error' no está en 'information' | 'warning'
```

- `onClose?.()` equivale a `if (onClose) { onClose(); }`: el operador `?.` solo llama a la función si no es `undefined` ni `null`.
- `React.FC<Props>` aparece en tutoriales antiguos: hoy **no se recomienda**, basta con tipar las props directamente.

Tipar el estado con useState

Declaración	Tipo resultante
`useState(true)`	`boolean`
`useState()`	`undefined`
`useState<boolean>()`	`boolean | undefined`
`useState<boolean>(true)`	`boolean`
`useState<Usuario | null>(null)`	`Usuario | null`
`useState<Tarea[]>([])`	`Tarea[]` (sin tipo, `[]` se infiere como `never[]`)

- Regla: **comprueba el tipo inferido**; si no es el adecuado, usa el **argumento genérico** `<T>`. Caso típico: estado que empieza vacío (`null`, `[]`) y luego recibe datos.

Tipar eventos y ampliar props de HTML

```
import { useState, type ChangeEvent, type FormEvent, type ComponentProps } from 'react';

function gestionarCambio(evento: ChangeEvent<HTMLInputElement>) { setNombre(evento.target.value); }
function gestionarEnvio(evento: FormEvent<HTMLFormElement>) { evento.preventDefault(); }

<form onSubmit={gestionarEnvio}>
  <input value={nombre} onChange={gestionarCambio} />
</form>

// Componente que acepta todos los atributos de <button> + una prop propia
type PropsBoton = ComponentProps<'button'> & { variante?: 'principal' | 'secundario' };
export function Boton({ variante = 'principal', ...resto }: PropsBoton) {
  return <button className={`boton-${variante}`} {...resto} />;
}
```

- **En línea** (`onChange={(e) => ...}`) TypeScript **infiere** el tipo del evento. **Extraído a una función aparte** hay que **anotarlo**. Truco: escribe el manejador en línea y pasa el cursor sobre `e`.

5. Estados y eventos en profundidad (UT 02.4)

5.1 Por qué hace falta el estado

Problema: un campo de correo que muestra un error al salir del campo (evento `blur`) si la dirección no es válida. Cuatro intentos:

Intento	Resultado
1. Componente estático	El mensaje se muestra siempre; no depende de lo que haga el usuario.
2. Tocar el DOM a mano (`querySelector`, `textContent`)	**No funciona**: el código se ejecuta antes de que React pinte los elementos (`null`). Además es **imperativo** (tú escribes los pasos) y React es **declarativo** (describes cómo se ve según los datos y React modifica el DOM).
3. Variable normal (`let mensajeError`)	La variable cambia (se ve con `console.log`) pero **la pantalla no se actualiza**: React no vigila variables normales y, además, `let mensajeError = ""` empezaría de cero en cada ejecución.
4. `useState` ✓	`setMensajeError(...)` guarda el valor entre renderizados **y avisa a React** de que debe volver a ejecutar el componente.

```
function EntradaCorreo() {
  const [mensajeError, setMensajeError] = useState("");
  function evaluarCorreo(evento: React.FocusEvent<HTMLInputElement>) {
    const correo = evento.target.value;
    setMensajeError(correo.trim() === "" || !correo.includes("@") ? "La dirección no es válida." : "");
  }
  return (
    <div>
      <input type="email" onBlur={evaluarCorreo} />
      {mensajeError && <p>{mensajeError}</p>}
    </div>
  );
}
```

> **Idea clave**
> El estado es un dato que, cuando cambia, **obliga a React a volver a ejecutar el componente** y a actualizar la pantalla si hace falta. Una variable normal no tiene ese poder. UI = f(estado).

5.2 Cómo se renderiza un componente

- **Renderizar = ejecutar la función del componente.** React la llama la primera vez y cada vez que cambia el estado (y también re-ejecuta a sus **hijos**).

Paso	Qué pasa
1	El usuario actúa (clic, escritura, envío).
2	El manejador de evento llama a `setEstado(valor)`.
3	React **guarda el valor y encola un renderizado**. Si el valor nuevo es igual al anterior (`Object.is`), **no renderiza**.
4	Nuevo renderizado: se vuelve a ejecutar el componente (y sus hijos).
5	React compara el JSX nuevo con el anterior (DOM virtual).
6	Se actualiza **solo el DOM que cambió**.

El estado es una “instantánea”

```
function Contador() {
  const [cuenta, setCuenta] = useState(0);
  function manejarClic() {
    console.log("antes:", cuenta);     // 0 en el primer clic
    setCuenta(cuenta + 1);
    console.log("después:", cuenta);   // sigue siendo 0, NO 1 !!
  }
  return <button onClick={manejarClic}>Pulsado {cuenta} veces</button>;
```

```
}
```

- Dentro de una misma ejecución, **el valor del estado no cambia** (es una foto). `setX` pide un renderizado **nuevo** en el que `useState` devolverá el valor actualizado. Por eso `cuenta` puede ser `const`.
- Con `setTimeout(() => console.log(cuenta), 3000)` y dos clics seguidos se imprimen `0` y `1` (cada manejador recuerda la foto de su ejecución).
- **Componente puro**: con las mismas props y estado devuelve siempre el mismo JSX y **no modifica nada externo durante el renderizado**. `StrictMode` (solo en desarrollo) ejecuta **dos veces** cada componente para detectarlo; un `console.log` duplicado no es un fallo.

5.3 Actualizar el estado correctamente: 3 reglas

Regla 1: el valor nuevo reemplaza al anterior (no se fusiona). Con objetos hay que copiar el resto de propiedades:

```
interface DatosAcceso { correo: string; contrasena: string; }
const [datos, setDatos] = useState<DatosAcceso>({ correo: "", contrasena: "" });

setDatos({ correo: "ana@correo.es" });                // Error de TS: falta 'contrasena' (en JS se perdería)
setDatos({ ...datos, correo: "ana@correo.es" });      // ✓ spread copia lo demás y sobrescribe correo
```

Regla 2: nunca modifiques el estado, crea una copia. React decide si re-renderiza **comparando referencias** (`Object.is`): si mutas el array/objeto y se lo devuelves, la referencia es la misma y no se actualiza nada.

```
tareas.push("Repasar"); setTareas(tareas);      // ✗ mismo array -> la pantalla NO se actualiza
setTareas([...tareas, "Repasar"]);              // ✓ array nuevo, referencia nueva
```

Operación	✗ Modifica (evitar)	✓ Crea una copia
Añadir	`push`, `unshift`	`[...lista, nuevo]`
Eliminar	`splice`, `pop`	`lista.filter(x => x.id !== id)`
Cambiar un elemento	`lista[0] = x`	`lista.map(x => x.id === id ? { ...x, hecha: true } : x)`
Ordenar	`sort`, `reverse`	`[...lista].sort()`
Objeto dentro de objeto	`usuario.direccion.ciudad = "Gijón"`	`{ ...usuario, direccion: { ...usuario.direccion, ciudad: "Gijón" } }`

Regla 3: si el valor nuevo depende del anterior, usa la forma funcional. React agrupa (*batching*) las actualizaciones de un mismo evento y todas leen la misma instantánea:

```
setCuenta(cuenta + 1); setCuenta(cuenta + 1); setCuenta(cuenta + 1);                    // ✗ suma solo 1
setCuenta(anterior => anterior + 1); setCuenta(anterior => anterior + 1); ...           // ✓ suma 3
setDatos((anteriores) => ({ ...anteriores, correo: nuevoCorreo }));                     // objetos
```

- Usa la forma funcional en **contadores, añadir a una lista, alternar un booleano**. Si el valor viene de otra fuente (texto de un input, valor fijo) pásalo directamente: `setNombre(e.target.value)`.
- ¿Varios `useState` o uno con objeto? **Varios** si los datos son independientes; **un objeto** si cambian juntos. Si un componente acumula muchos estados, divídelo en componentes más pequeños.

5.4 Qué merece ser estado

Pregunta (en orden)	Si la respuesta es SÍ →
¿Cambia con el uso (tras un evento)? **NO**	**Constante** (fuera del componente).
¿Llega por props?	Es una **prop**: no se copia al estado.
¿Se puede calcular a partir de otros datos?	**Dato derivado**: variable normal en el cuerpo del componente.
Ninguna de las anteriores	**Estado** (`useState`).

```
function ListaProductos({ productos }: { productos: Producto[] }) {
```

```
  const [busqueda, setBusqueda] = useState("");              // ESTADO: solo el criterio
  const visibles = productos.filter((p) =>                   // DERIVADO: se recalcula en cada render
    p.nombre.toLowerCase().includes(busqueda.toLowerCase()));
  ...
}
```

- Guarda en estado **lo mínimo imprescindible** y calcula el resto (así no hay datos que se desincronicen). Los errores de validación también son **datos derivados**.
- **Error 1: duplicar información** (`texto` y `largo` en dos estados; `largo = texto.length` basta).
- **Error 2: copiar una prop al estado** (`useState(nombre)` solo lee la prop **la primera vez**; si el padre la cambia, no se entera). Usa la prop directamente.

5.5 Eventos en profundidad

Propiedad / método del evento	Qué es
`target`	Elemento donde **se originó** el evento (`e.target.value` en un input).
`currentTarget`	Elemento **al que está unido el manejador**.
`type`, `key`	Nombre del evento / tecla pulsada (`"Enter"`, `"Escape"`).
`preventDefault()`	Cancela el comportamiento **por defecto del navegador** (recargar al enviar un form, navegar al pulsar un `<a>`).
`stopPropagation()`	Corta la **subida (burbujeo)** del evento a los elementos padre.

Evento	Cuándo se dispara
`onClick`	Al hacer clic.
`onChange`	En React, **en cada pulsación** (en el DOM nativo eso es `input`). Si solo quieres reaccionar al terminar, usa `onBlur`.
`onBlur` / `onFocus`	Al perder / ganar el foco.
`onKeyDown`	Al pulsar una tecla (atajos, enviar con Enter).
`onSubmit`	Al enviar el formulario (botón `type="submit"` o Enter en un campo).
`onMouseEnter` / `onMouseLeave`	El ratón entra / sale.

```
// Burbujeo: el clic del botón sube al div. stopPropagation lo evita.
function borrar(e: React.MouseEvent<HTMLButtonElement>) {
  e.stopPropagation();          // sin esto, también se ejecutaría abrirDetalle del div
  console.log("Borrando", id);
}
<div onClick={abrirDetalle}><button onClick={borrar}>Borrar</button></div>

// Pasar un dato al manejador: función flecha
<button onClick={() => quitar(a.id)}>Quitar</button>      // ✓
<button onClick={quitar(a.id)}>Quitar</button>            // ✗ se ejecuta al renderizar

// Teclado
onKeyDown={(e) => {
  if (e.key === "Enter") console.log("Buscando", texto);
  if (e.key === "Escape") setTexto("");
}}
```

- Orden en el burbujeo: primero el manejador del **botón**, después el del **div**. No confundir `preventDefault()` (cancela la acción del navegador) con `stopPropagation()` (corta la subida).
- Convenio de nombres: manejadores `manejarXxx` / `gestionarXxx`; props de evento propio `alXxx` / `onXxx`.

5.6 Formularios con campos controlados

- Un campo es **controlado** cuando React gobierna su valor: el input **muestra el estado** (`value`) y **actualiza el estado** al escribir (`onChange`) → enlace bidireccional. La fuente de verdad es el estado, no el DOM.

- `value` **sin** `onChange` deja el campo de solo lectura (React avisa). Para un valor inicial editable usa `defaultValue`.
- `htmlFor` es el `for` de `<label>` en React (`for` es palabra reservada de JS); une la etiqueta con el campo por `id` y mejora la accesibilidad.

Campo	Propiedad que enlaza el estado	Qué se lee en el evento	Tipo del estado
Texto, correo, contraseña, `textarea`	`value`	`e.target.value`	`string`
Casilla (`checkbox`)	`checked`	`e.target.checked`	`boolean`
Lista desplegable (`select`)	`value`	`e.target.value`	`string`

```
function AltaBoletin() {
  const [correo, setCorreo] = useState("");
  const [acepta, setAcepta] = useState(false);
  const [enviado, setEnviado] = useState(false);

  const errorCorreo = correo === "" ? "El correo es obligatorio"       // DERIVADO, no es estado
                    : !correo.includes("@") ? "Falta el símbolo @" : null;

  function manejarEnvio(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();                // evita la recarga de la página
    setEnviado(true);
  }
  if (enviado) return <p>¡Gracias por suscribirte, {correo}!</p>;       // return anticipado

  return (
    <form onSubmit={manejarEnvio}>
      <label htmlFor="correo">Tu correo</label>
      <input id="correo" type="email" value={correo} onChange={(e) => setCorreo(e.target.value)} />
      {errorCorreo && <p role="alert">{errorCorreo}</p>}
      <input id="acepta" type="checkbox" checked={acepta} onChange={(e) => setAcepta(e.target.checked)} />
      <label htmlFor="acepta">Acepto los términos</label>
      <button type="submit" disabled={!acepta || correo === ""}>Suscribirme</button>
    </form>
  );
}
```

5.7 Elevar el estado (lifting state up)

- **Problema**: dos componentes hermanos necesitan el mismo dato, pero en React **los datos fluyen de padre a hijo**; un componente no puede leer el estado de un hermano.
- **Solución**: colocar el estado en el **ancestro común más cercano**; el **valor baja por props** y las **funciones para modificarlo bajan como eventos propios** (los datos bajan, los eventos suben).

```
interface PropsBarraBusqueda { termino: string; alCambiarTermino: (nuevoTermino: string) => void; }

function BarraBusqueda({ termino, alCambiarTermino }: PropsBarraBusqueda) {
  return <input type="search" value={termino} onChange={(e) => alCambiarTermino(e.target.value)} />;
}
function Resumen({ terminoActual }: { terminoActual: string }) {
  return <p>Buscando: {terminoActual || "(nada todavía)"}</p>;
}
function App() {
  const [termino, setTermino] = useState("");        // el estado vive en el ancestro común
  return (<>
    <BarraBusqueda termino={termino} alCambiarTermino={setTermino} />
    <Resumen terminoActual={termino} />
  </>);
}
```

- Pasos: 1) identificar qué componentes necesitan el dato; 2) localizar su ancestro común; 3) mover allí el `useState`; 4) pasar el **valor** por props y una **función** a los hijos que lo modifican; 5) convertir los hijos en componentes que **muestran lo que reciben**, sin estado propio para ese dato.
- Si hay muchos niveles (*prop drilling*) existen **Context** y bibliotecas como **Zustand** (temas futuros).

5.8 Tipado de eventos (anexo)

- Molde: `React.MouseEvent<HTMLButtonElement>` = **tipo de evento** + **elemento al que se une el manejador** (genérico). El genérico determina el tipo de `e.currentTarget` (y de `e.target` en `ChangeEvent` y `FocusEvent`).
- En `MouseEvent`/`KeyboardEvent`, `e.target` es un `EventTarget` genérico (**sin** `value`): usa `currentTarget` o el evento correcto.

Evento JSX	Tipo del evento	Tipo del manejador
`onClick`	`MouseEvent<HTMLButtonElement>`	`MouseEventHandler<...>`
`onChange` (input)	`ChangeEvent<HTMLInputElement>`	`ChangeEventHandler<HTMLInputElement>`
`onChange` (textarea)	`ChangeEvent<HTMLTextAreaElement>`	`ChangeEventHandler<HTMLTextAreaElement>`
`onChange` (select)	`ChangeEvent<HTMLSelectElement>`	`ChangeEventHandler<HTMLSelectElement>`
`onSubmit`	`FormEvent<HTMLFormElement>`	`FormEventHandler<HTMLFormElement>`
`onKeyDown`	`KeyboardEvent<HTMLInputElement>`	`KeyboardEventHandler<HTMLInputElement>`
`onBlur` / `onFocus`	`FocusEvent<HTMLInputElement>`	`FocusEventHandler<HTMLInputElement>`
`onMouseEnter`	`MouseEvent<HTMLDivElement>`	`MouseEventHandler<HTMLDivElement>`

- Regla de nombres: el tipo del manejador es el del evento con el sufijo `Handler`.
- Tres formas de tipar: anotando el parámetro; anotando la constante con el tipo `Handler` (el parámetro se infiere); o **en línea** (todo se infiere).
- **Props que reciben manejadores**: *Estilo A* pasar el evento completo (`alPulsar: React.MouseEventHandler<HTMLButtonElement>`); *Estilo B (recomendado)* pasar solo el dato (`alCambiar: (nuevoValor: string) => void`), que desacopla al padre del DOM.
- Un manejador para varios tipos de elemento: unión en el genérico, `ChangeEvent<HTMLInputElement | HTMLSelectElement>`; solo puedes usar propiedades que tengan **todos** (p. ej. `checked` no existe en `select`).
- Para campos numéricos: `e.target.valueAsNumber`.

```
// Un manejador, varios campos (usa el atributo name)
function manejarCampo(e: ChangeEvent<HTMLInputElement | HTMLSelectElement>) {
  const { name, value } = e.target;
  setFicha((anterior) => ({ ...anterior, [name]: value }));
}
```

Mensaje de TypeScript	Causa y solución
`Parameter 'e' implicitly has an 'any' type`	Función aparte sin anotar: tipar el parámetro.
`Property 'value' does not exist on type 'EventTarget'`	Se lee `e.target.value` en un evento que no es `ChangeEvent`: usar `ChangeEvent` o `currentTarget`.
`Type 'ChangeEventHandler' is not assignable...`	El tipo del manejador no coincide con el evento de la propiedad.
`MouseEvent` del DOM frente al de React	Usar `React.MouseEvent` (o `import type` de `react`), no el global.
`Property 'checked' does not exist` (select)	`checked` solo existe en `input` checkbox.

6. Errores frecuentes y buenas prácticas

Error	Síntoma	Corrección
`onClick={manejar()}`	La función se ejecuta al renderizar.	`onClick={manejar}` o `onClick={() => manejar(x)}`
Variable normal (`let`) en vez de estado	La pantalla no cambia.	`useState`
Modificar el estado (`push`, `obj.x = 1`)	La interfaz no se actualiza.	Crear una copia: `[...lista, x]`, `{ ...obj, x: 1 }`
Leer el estado justo después de `setX`	Se ve el valor antiguo.	El estado es una instantánea; usa una variable local con el valor nuevo.
`setX(x + 1)` varias veces seguidas	Solo suma 1.	`setX(anterior => anterior + 1)`
Pasar un objeto parcial a `setX`	Se pierden propiedades (error de TS).	`setX({ ...anterior, campo: nuevo })`
Guardar en estado algo que se puede calcular	Datos que dejan de coincidir.	Calcularlo en el cuerpo del componente.
Copiar una prop al estado	No se actualiza cuando el padre la cambia.	Usar la prop directamente.
Llamar a `setX` durante el renderizado	Bucle infinito (*Too many rerenders*).	Llamarlo solo desde manejadores de eventos.
`value` sin `onChange`	Campo de solo lectura.	Añadir `onChange` o usar `defaultValue`.
`useState([])` sin tipo	TS infiere `never[]`.	`useState<Tarea[]>([])`
Olvidar `e.preventDefault()` en un formulario	La página se recarga al enviar.	Llamarlo al principio de `manejarEnvio`.
Componente con minúscula	No se renderiza (se trata como HTML).	Empezar por mayúscula.
Importar un tipo sin `type`	Error por `verbatimModuleSyntax`.	`import { useState, type ReactNode } from 'react'`

Buenas prácticas

- **Estado mínimo**: guarda solo lo imprescindible y calcula el resto.
- **Estado inmutable** y forma funcional cuando dependas del valor anterior.
- **Deja que TypeScript infiera**; anota cuando haga falta (estado que empieza en `null`/`[]`, manejadores extraídos, props).
- **Nunca** `any`; si no sabes el tipo, `unknown` y estrechar.
- **Estado cerca**: en el componente más bajo que lo necesite; súbelo solo cuando haya que compartirlo.
- **Nombres claros**: `[valor, setValor]`, `manejarXxx`, `alXxx`/`onXxx` para props de función.

7. Preguntas de repaso con respuesta

1. ¿Por qué una variable normal no sirve como estado?

→ React no vigila las variables normales, así que no sabe que ha cambiado y no vuelve a ejecutar el componente; además se reinicia en cada ejecución. Hace falta `useState`.

2. En un clic, `setCuenta(cuenta + 1); console.log(cuenta);` con `cuenta` valiendo 3. ¿Qué imprime y cuánto vale en el siguiente renderizado?

→ Imprime 3 (el estado es una instantánea) y en el siguiente renderizado valdrá 4.

3. ¿Diferencia entre `onClick={borrar}` y `onClick={borrar()}`?

→ `borrar` pasa la función para que React la llame cuando ocurra el evento; `borrar()` la ejecuta al renderizar y pasa su resultado.

4. ¿Por qué `tareas.push("x"); setTareas(tareas);` no actualiza la pantalla?

→ Se modifica el mismo array: la referencia no cambia y React (que compara referencias con `Object.is`) no detecta cambio. Hay que hacer `setTareas([...tareas, "x"])`.

5. Una tarjeta con `onClick` contiene un botón con otro `onClick`. ¿Qué pasa al pulsar el botón y cómo se evita el de la tarjeta?

→ Se ejecuta primero el del botón y después, por burbujeo, el de la tarjeta. Se evita con `e.stopPropagation()` en el manejador del botón.

6. ¿Diferencia entre `target` y `currentTarget`?

→ `target` es el elemento donde se originó el evento; `currentTarget` es el elemento que tiene el manejador.

7. ¿Cuándo usar la forma funcional de `setX`?

→ Cuando el valor nuevo depende del anterior (contadores, añadir a una lista, alternar un booleano).

8. ¿Cómo comparten estado dos componentes hermanos?

→ Elevando el estado al ancestro común: el valor baja por props y una función (evento propio) sube los cambios.

9. ¿Qué tipo infiere TS en `let mensaje = 'Hola';`?

→ `string`.

10. ¿Por qué no hay error de tipo en `let datos: any = {nombre:'Ana'}; datos.edad.toFixed(2)`?

→ Porque `any` desactiva la comprobación de tipos; el fallo solo aparecerá en ejecución.

11. ¿Qué falla en `function f(valor: unknown){ console.log(valor.nombre); }` y cómo se corrige?

→ No se pueden usar propiedades de un `unknown` sin estrechar el tipo: comprobar `typeof valor === 'object' && valor !== null && 'nombre' in valor` antes.

12. ¿Ventaja de `type?: 'information' | 'warning'` frente a `type?: string`?

→ TS solo acepta los dos valores válidos, detecta erratas y el editor autocompleta. Con `string` aceptaría cualquier cadena.

13. ¿Qué tipo tienen `useState(true)` y `useState<boolean>()`?

→ `boolean` y `boolean | undefined` respectivamente.

14. ¿Por qué falla `import { useState, ReactNode } from 'react'` en la plantilla react-ts?

→ Porque `verbatimModuleSyntax` obliga a importar los tipos con `type`: `import { useState, type ReactNode } from 'react'`.

15. ¿Diferencia entre `npm run dev` y `npm run build`?

→ `dev` arranca el servidor de desarrollo (localhost:5173) con recarga; `build` genera en `dist` la versión de producción (minificada, empaquetada y con tree-shaking).

16. ¿Para qué sirve `useState` y qué devuelve?

→ Define una variable de estado; devuelve una tupla [valor actual, función para actualizarlo]. Al llamar a la función, React vuelve a renderizar.

PARTE 2

Ejercicios de repaso

EJER_Repaso examen · React + TypeScript · estado, eventos y formularios básicos

Cómo usar esta hoja

- Realizar todos con **React y TypeScript**: proyecto con Vite (plantilla React + TypeScript) y **un componente por ejercicio**.
- Nombres **en español** para variables, funciones, props e interfaces.
- **Tipa siempre** el estado (`useState<tipo>`), las props y los eventos. **No se admite** `any`**.**
- **Nunca mutes el estado**: copia con spread, `map` o `filter`. Los datos que se pueden calcular **no se guardan en estado**.

```
npm create vite@latest repaso-examen -- --template react-ts
cd repaso-examen
npm install
npm run dev
```

App.tsx para probar todos los componentes

```
import { InterruptorLuz } from './InterruptorLuz';
import { ContadorLimitado } from './ContadorLimitado';
import { ListaCompra } from './ListaCompra';
import { BuscadorPeliculas } from './BuscadorPeliculas';

function App() {
  return (
    <>
      <InterruptorLuz />
      <hr />
      <ContadorLimitado />
      <hr />
      <ListaCompra />
      <hr />
      <BuscadorPeliculas />
    </>
  );
}

export default App;
```

Ejercicio 1. Interruptor de luz

Enunciado
Crea `InterruptorLuz` con un botón que alterna entre luz encendida y apagada. Estado `encendida` (`boolean`). El texto del botón cambia: “Encender” o “Apagar”. Un recuadro cambia de color de fondo (amarillo o gris oscuro) según el estado. Pista: `setEncendida(!encendida)`.

```
// src/InterruptorLuz.tsx
import { useState } from 'react';

export function InterruptorLuz() {
  const [encendida, setEncendida] = useState<boolean>(false);

  return (
    <div>
      <div
        style={{
          width: 200,
          height: 120,
          borderRadius: 8,
          backgroundColor: encendida ? 'yellow' : '#333',   // color derivado del estado
        }}
      />
      <button onClick={() => setEncendida(!encendida)}>
        {encendida ? 'Apagar' : 'Encender'}
      </button>
    </div>
  );
}
```

Por qué está así
Solo hay **un estado booleano**; el texto del botón y el color son **derivados** del estado y se calculan en el JSX. Alternativa con forma funcional (más segura): `setEncendida((anterior) => !anterior)`. `useState<boolean>(false)` ya se infiere como `boolean`, pero el enunciado pide tiparlo explícitamente.

Ejercicio 2. Contador con límites

Enunciado
Crea `ContadorLimitado` con tres botones: sumar, restar y reiniciar. El valor está siempre entre 0 y 10. El botón de sumar se desactiva (`disabled`) al llegar a 10 y el de restar al llegar a 0. Sumar y restar usan la forma funcional `setCuenta(anterior => anterior + 1)`. Extra: muestra “Máximo alcanzado” cuando valga 10 (dato derivado, no estado).

```
// src/ContadorLimitado.tsx
import { useState } from 'react';

const MINIMO = 0;
const MAXIMO = 10;

export function ContadorLimitado() {
  const [cuenta, setCuenta] = useState<number>(MINIMO);

  // Datos DERIVADOS: se calculan en cada renderizado, no son estado
  const enMaximo = cuenta >= MAXIMO;
  const enMinimo = cuenta <= MINIMO;

  return (
    <div>
      <p>Valor: {cuenta}</p>
      <button onClick={() => setCuenta((anterior) => anterior + 1)} disabled={enMaximo}>
        Sumar
      </button>
      <button onClick={() => setCuenta((anterior) => anterior - 1)} disabled={enMinimo}>
        Restar
      </button>
      <button onClick={() => setCuenta(MINIMO)}>Reiniciar</button>
      {enMaximo && <p>Máximo alcanzado</p>}
    </div>
  );
}
```

Por qué está así
`disabled` impide que el valor se salga de [0, 10] porque no se puede pulsar fuera de límites. Se usa la **forma funcional** porque el nuevo valor depende del anterior. “Máximo alcanzado” se calcula con `cuenta >= MAXIMO` (**dato derivado**): un segundo estado `useState` para esto sería el error de “duplicar información”. Reiniciar usa valor directo (`0`) porque no depende del anterior.

Ejercicio 3. Lista de la compra

Enunciado
Crea `ListaCompra`. `interface Producto { id: number; nombre: string; comprado: boolean }`. Un campo de texto y un botón “Añadir” (no se añaden textos vacíos). Cada producto tiene una casilla para marcarlo como comprado y un botón “Quitar”. Los comprados se muestran tachados. Muestra “Pendientes: N” calculado en el renderizado. Prohibido `push`, `splice` y asignar propiedades: usa `[...lista, nuevo]`, `map` y `filter`.

```
// src/ListaCompra.tsx
import { useState, type ChangeEvent, type FormEvent } from 'react';

interface Producto {
  id: number;
  nombre: string;
  comprado: boolean;
}

export function ListaCompra() {
  const [productos, setProductos] = useState<Producto[]>([]);   // [] necesita tipo explícito
  const [texto, setTexto] = useState<string>('');

  // Dato DERIVADO: no se guarda en estado
  const pendientes = productos.filter((producto) => !producto.comprado).length;

  function manejarCambioTexto(e: ChangeEvent<HTMLInputElement>) {
    setTexto(e.target.value);
  }

  function manejarEnvio(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();                       // evita recargar la página
    const nombre = texto.trim();
    if (nombre === '') return;                // no se añaden textos vacíos

    const nuevo: Producto = { id: Date.now(), nombre, comprado: false };
    setProductos((anteriores) => [...anteriores, nuevo]);   // copia + nuevo
    setTexto('');                                            // limpia el campo controlado
  }

  function alternarComprado(id: number) {
    setProductos((anteriores) =>
      anteriores.map((producto) =>
        producto.id === id ? { ...producto, comprado: !producto.comprado } : producto,
      ),
    );
  }

  function quitar(id: number) {
    setProductos((anteriores) => anteriores.filter((producto) => producto.id !== id));
  }

  return (
    <div>
      <form onSubmit={manejarEnvio}>
        <input value={texto} onChange={manejarCambioTexto} placeholder="Nuevo producto" />
        <button type="submit">Añadir</button>
      </form>

      <p>Pendientes: {pendientes}</p>

      <ul>
        {productos.map((producto) => (
          <li key={producto.id}>
            <input
              type="checkbox"
              checked={producto.comprado}
              onChange={() => alternarComprado(producto.id)}
            />
            <span style={{ textDecoration: producto.comprado ? 'line-through' : 'none' }}>
              {producto.nombre}
            </span>
```

```
            <button onClick={() => quitar(producto.id)}>Quitar</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Por qué está así
Dos estados (`productos` y `texto` del campo controlado); `pendientes` es **derivado**. **Añadir** → `[...anteriores, nuevo]`; **marcar** → `map` devolviendo una copia del producto con `{ ...producto, comprado: !producto.comprado }` (no se asigna la propiedad); **quitar** → `filter`. `key={producto.id}` es obligatorio en listas. El `useState<Producto[]>([])` lleva tipo porque `[]` se inferiría como `never[]`. `e.preventDefault()` evita la recarga al enviar el formulario.

Ejercicio 4. Buscador de películas

Enunciado
Crea `BuscadorPeliculas` con un array fijo de al menos 8 películas (`titulo`, `anio`, `genero`). Estado: solo `busqueda` (texto) y `genero` (un `select` con "Todos" y los géneros). La lista filtrada y el contador "N resultados" son datos derivados. Al pulsar `Escape` en el campo de búsqueda se borra el texto (`React.KeyboardEvent<HTMLInputElement>`). Si no hay resultados, muestra "Sin resultados". Extra: botón "Limpiar filtros".

```
// src/BuscadorPeliculas.tsx
import { useState, type ChangeEvent, type KeyboardEvent } from 'react';

interface Pelicula {
  titulo: string;
  anio: number;
  genero: string;
}

// Array fijo: constante FUERA del componente (no cambia, no es estado)
const PELICULAS: Pelicula[] = [
  { titulo: 'El Padrino', anio: 1972, genero: 'Drama' },
  { titulo: 'Matrix', anio: 1999, genero: 'Ciencia ficción' },
  { titulo: 'Interstellar', anio: 2014, genero: 'Ciencia ficción' },
  { titulo: 'Pulp Fiction', anio: 1994, genero: 'Crimen' },
  { titulo: 'Toy Story', anio: 1995, genero: 'Animación' },
  { titulo: 'El Rey León', anio: 1994, genero: 'Animación' },
  { titulo: 'Gladiator', anio: 2000, genero: 'Acción' },
  { titulo: 'Origen', anio: 2010, genero: 'Ciencia ficción' },
  { titulo: 'Seven', anio: 1995, genero: 'Crimen' },
  { titulo: 'Titanic', anio: 1997, genero: 'Drama' },
];

// Opciones del select: también derivadas de los datos y constantes
const GENEROS: string[] = ['Todos', ...new Set(PELICULAS.map((p) => p.genero))];

export function BuscadorPeliculas() {
  const [busqueda, setBusqueda] = useState<string>('');
  const [genero, setGenero] = useState<string>('Todos');

  // DERIVADOS: se recalculan en cada renderizado
  const visibles = PELICULAS.filter(
    (pelicula) =>
      pelicula.titulo.toLowerCase().includes(busqueda.toLowerCase()) &&
      (genero === 'Todos' || pelicula.genero === genero),
  );
  const numeroResultados = visibles.length;

  function manejarBusqueda(e: ChangeEvent<HTMLInputElement>) {
    setBusqueda(e.target.value);
  }

  function manejarGenero(e: ChangeEvent<HTMLSelectElement>) {
    setGenero(e.target.value);
  }

  function manejarTecla(e: KeyboardEvent<HTMLInputElement>) {
    if (e.key === 'Escape') {
      setBusqueda('');
    }
  }

  function limpiarFiltros() {
    setBusqueda('');
    setGenero('Todos');
  }

  return (
    <div>
      <input
        type="search"
        placeholder="Buscar por título"
```

```
        value={busqueda}
        onChange={manejarBusqueda}
        onKeyDown={manejarTecla}
      />

      <select value={genero} onChange={manejarGenero}>
        {GENEROS.map((g) => (
          <option key={g} value={g}>
            {g}
          </option>
        ))}
      </select>

      <button onClick={limpiarFiltros}>Limpiar filtros</button>

      <p>{numeroResultados} resultados</p>

      {numeroResultados === 0 ? (
        <p>Sin resultados</p>
      ) : (
        <ul>
          {visibles.map((pelicula) => (
            <li key={pelicula.titulo}>
              {pelicula.titulo} ({pelicula.anio}) · {pelicula.genero}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

Por qué está así
Solo hay **dos estados** (`busqueda` y `genero`). La lista `visibles` y `numeroResultados` son **datos derivados**; no existe un estado "peliculasFiltradas". Los dos campos son **controlados** (`value` + `onChange`). Cada manejador extraído lleva su tipo: `ChangeEvent<HTMLInputElement>`, `ChangeEvent<HTMLSelectElement>` y `KeyboardEvent<HTMLInputElement>`. `Escape` vacía la búsqueda con `setBusqueda('')`. El renderizado condicional ternario muestra "Sin resultados" si no hay coincidencias.

Checklist antes de dar por bueno un ejercicio

- ¿El estado está tipado (`useState<tipo>`) y no hay ningún `any`?
- ¿Props, manejadores extraídos y eventos tienen su tipo (`ChangeEvent`, `FormEvent`, `KeyboardEvent`)?
- ¿Evito `push`, `splice` y asignar propiedades? ¿Uso `[...lista, x]`, `map`, `filter` y `{ ...obj, campo }`?
- ¿Lo que se puede calcular (contadores, listas filtradas, mensajes) es una variable normal y no un `useState`?
- ¿Uso la forma funcional `setX(anterior => ...)` cuando el valor depende del anterior?
- ¿Las listas llevan `key` y los inputs controlados llevan `value` + `onChange`?
- ¿Los manejadores se pasan como `onClick={funcion}` o `onClick={() => funcion(x)}` (nunca `funcion()`)?